

Unit 1: Computational Thinking & Algorithmic Logic • Topic 4

Python Basics & Syntax

Variables, data types, type casting, operators, indentation, comments, and input/output in Python.

With problem analysis, algorithm design, flowcharts and pseudocode covered, it is time to translate logic into actual **Python** code. This topic covers the essential building blocks of Python syntax: variables, data types, type casting, operators, indentation, comments, and input/output.

4.1 Variables

A **variable** is a named location in memory used to store a value that can change during program execution. In Python, a variable is created the moment you assign a value to it — there is no separate declaration step.

```

age = 18          # variable 'age' now refers to the integer 18
price = 499.50    # variable 'price' now refers to a float
name = "Aarav"    # variable 'name' now refers to a string
age = 19          # the SAME variable can later refer to a new value

```

Rules for Naming Variables

- May contain letters, digits and underscores (_) only — no spaces or special symbols.
- Cannot start with a digit (1value is invalid, value1 is fine).
- Cannot be a Python keyword (e.g. for, if, class).
- Case-sensitive: Marks and marks are two different variables.
- By convention, use lowercase with underscores for readability, e.g. total_marks.

Python is **dynamically typed**: you do not declare a variable's type in advance — the interpreter figures it out automatically from the value assigned, and the same variable name can be reassigned to a value of a different type later.

4.2 Data Types

Every value in Python has a **data type**, which determines what kind of data it holds and what operations can be performed on it. The `type()` function reveals a value's type.

Type	Example	Description
int	10, -3, 2026	Whole numbers, positive or negative
float	3.14, -0.5, 2.0	Numbers with a decimal point
complex	3 + 4j	Numbers with a real and imaginary part
str	"Hello", 'Python'	Text, enclosed in single or double quotes
bool	True, False	Logical / truth values
list	[1, 2, 3]	Ordered, changeable collection (covered in Unit 4)
tuple	(1, 2, 3)	Ordered, unchangeable collection (covered in Unit 3)
dict	{"a": 1}	Key-value pairs (covered in Unit 4)

Type	Example	Description
NoneType	None	Represents 'no value'

```
x = 10
y = 3.14
z = "Python"
flag = True
print(type(x))      # <class 'int'>
print(type(y))      # <class 'float'>
print(type(z))      # <class 'str'>
print(type(flag))   # <class 'bool'>
```

4.3 Type Casting (Type Conversion)

Type casting is converting a value from one data type to another. Python supports two kinds:

- **Implicit type casting** – Python automatically converts a smaller/compatible type to a larger one during an operation, without the programmer's intervention, e.g. $\text{int} + \text{float} \rightarrow \text{float}$.
- **Explicit type casting** – the programmer manually converts a value using functions such as `int()`, `float()`, `str()`, and `bool()`.

```
# Implicit casting
num_int = 10
num_float = 2.5
result = num_int + num_float      # int is implicitly converted to float
print(result, type(result))      # 12.5 <class 'float'>

# Explicit casting
age_str = "20"
age_int = int(age_str)            # string -> int
price = float("499.5")           # string -> float
count_str = str(25)              # int -> string
print(age_int + 1)               # 21
```

COMMON PITFALL: `int("12.5")` raises a `ValueError` — `int()` cannot parse a decimal point directly from a string. Convert to float first: `int(float("12.5"))`.

4.4 Operators

Operators are special symbols that perform operations on values (called operands).

Arithmetic Operators

Operator	Meaning	Example (a=7, b=2)
+	Addition	$a + b \rightarrow 9$
-	Subtraction	$a - b \rightarrow 5$
*	Multiplication	$a * b \rightarrow 14$
/	Division (always returns a float)	$a / b \rightarrow 3.5$
//	Floor division (drops the decimal part)	$a // b \rightarrow 3$

Operator	Meaning	Example (a=7, b=2)
%	Modulus (remainder)	a % b → 1
**	Exponentiation (power)	a ** b → 49

Relational (Comparison) Operators

These compare two values and always produce a bool result (True/False): == (equal to), != (not equal to), >, <, >=, <=.

Logical Operators

and (True only if both conditions are True), or (True if at least one condition is True), not (reverses True/False). These are covered in more depth with conditional statements in Unit 2.

Assignment Operators

Operator	Example	Equivalent to
=	x = 5	Assign 5 to x
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3

Other Operator Categories (brief)

- **Identity operators** – is, is not – check whether two variables point to the same object in memory.
- **Membership operators** – in, not in – check whether a value exists within a sequence, e.g. 'a' in 'cat' → True.
- **Bitwise operators** – &, |, ^, ~, <<, >> – operate directly on the binary bits of integers.

Operator Precedence (highest to lowest, simplified)

** > unary +x, -x > *, /, //, % > +, - > comparison operators (==, !=, >, < ...) > not > and > or. Parentheses () always override precedence and should be used freely for clarity.

4.5 Indentation

Unlike many languages that use curly braces { } to mark a block of code, **Python uses indentation (whitespace at the start of a line)** to define blocks — such as the body of a loop, an if statement, or a function. Indentation in Python is not just for readability; it is **mandatory syntax**.

```
if 5 > 2:
    print("Five is greater than two!") # indented -> part of the if-block
print("This runs regardless")        # not indented -> outside the if-block
```

- Use a consistent number of spaces (4 spaces is the standard convention) for each indent level.
- Never mix tabs and spaces in the same file — this causes an IndentationError.

- All statements within the same block must be indented by exactly the same amount.

COMMON ERROR: IndentationError: expected an indented block — this means Python expected a new indented block (e.g. after a colon :) but found none.

4.6 Comments

Comments are notes in the source code that the Python interpreter ignores completely — they exist purely to make code understandable to humans.

```
# This is a single-line comment
x = 5    # comments can also follow code on the same line

"""
This is a multi-line string, often used as a
multi-line comment or a docstring describing
what a function/module does.
"""
```

A **docstring** is a special string placed as the first statement inside a function, class, or module to document what it does; it is accessible via the built-in `help()` function.

4.7 Input and Output

The `input()` Function

`input()` pauses the program, waits for the user to type something and press Enter, and returns it **always as a string** — even if the user types a number, it must be explicitly cast if numeric operations are needed.

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))    # cast to int for arithmetic
print("Next year you will be", age + 1)
```

The `print()` Function

`print()` displays output on the screen. It can take multiple values separated by commas, and two useful keyword arguments: `sep` (separator between values, default is a space) and `end` (what to print after the last value, default is a newline).

```
print("Sum is", 15)                # Sum is 15
print("a", "b", "c", sep="-")      # a-b-c
print("Loading", end="...")        # no newline after this
print("Done")                     # Loading...Done
```

Formatted Output with f-strings

An **f-string** (formatted string literal), written as `f"..."`, lets you embed variables and expressions directly inside a string using curly braces `{ }` — the modern, preferred way to build formatted output in Python.

```
name = "Diya"
marks = 91.6667
print(f"Hello {name}, your average is {marks:.2f}%")
# Output: Hello Diya, your average is 91.67%
```

The `.2f` inside the braces is a **format specifier** that rounds a float to 2 decimal places. Older equivalents are the `%` operator (`"%.2f" % marks`) and the `.format()` method (`"{: .2f}".format(marks)`).

Practice Questions

A. Multiple Choice Questions

- Which of the following is an invalid Python variable name?
(A) total_marks (B) _value (C) 2ndValue (D) value2
- What is the data type of the value produced by input()?
(A) int (B) float (C) str (D) bool
- Which operator returns the remainder of a division?
(A) / (B) // (C) % (D) **
- What does 17 // 4 evaluate to in Python?
(A) 4.25 (B) 4 (C) 1 (D) 4.0
- Which symbol does Python use to define a block of code (instead of curly braces)?
(A) Semicolon (B) Indentation (C) Parentheses (D) Colon alone, indentation optional
- Which function explicitly converts a string to an integer?
(A) str() (B) float() (C) int() (D) bool()
- In print('a','b', sep='-'), what is displayed?
(A) a b (B) a-b (C) ab (D) a, b

B. True / False

- Python variables must be declared with a data type before assigning a value.
- The expression 5 + 2.0 results in a float due to implicit type casting.
- A single-line comment in Python starts with //.
- f-strings allow variables to be embedded directly inside a string using curly braces.
- Indentation in Python is optional and only affects readability.

C. Short Answer Questions

- List any four rules for naming variables in Python.
- Differentiate between implicit and explicit type casting, with one example of each.
- What is the difference between the / and // operators? Give an example.
- Why does Python require int() casting after using input() for numeric operations?
- What is the purpose of the sep and end parameters of the print() function?

D. Long Answer / Descriptive Questions

- Explain Python's basic data types (int, float, str, bool) with one example of each, and show how the type() function can be used to verify them.
- Discuss the role of indentation in Python with a code example, and explain what error occurs when indentation is inconsistent.
- Explain arithmetic, relational and assignment operators in Python with suitable examples for each category.

E. Programming Problems

- 21.** Write a Python program that takes the radius of a circle as input and prints its area and circumference, formatted to 2 decimal places using an f-string.
- 22.** Write a Python program that takes two numbers as input and prints the result of +, -, *, /, //, %, and ** on them.
- 23.** Write a Python program that asks the user for their name and age (as input), and prints a message stating their name and the year they will turn 60, using an f-string.
- 24.** Write a Python program to swap the values of two variables (a) using a temporary variable and (b) without using a temporary variable.

Answer Key (A & B): MCQ: 1-C, 2-C, 3-C, 4-B, 5-B, 6-C, 7-B | T/F: 8-False, 9-True, 10-False, 11-True, 12-False